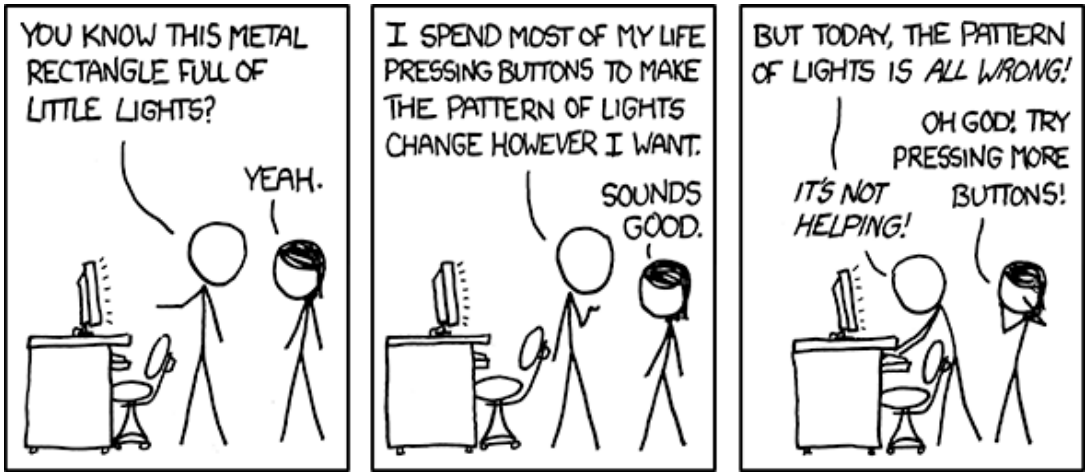
# Reference and Practice Handout

Lecture-07: Instructions Representations

Computer Organization & Assembly Language Programming
CS 2160 || Fall 2026
Instructor: Himon Thakur




Computer Problems (Randall Munroe; CC BY-NC 2.5; https://xkcd.com/722/)

This document is a supplementary resource alongside course materials and class lectures. You are expected to go through and utilize it by yourself. You may read a section, then try solving the short exercises before moving on. Answers are provided at the end.

This handout will cover:

- Why the same **32 bits** can mean a *number*, *text*, or an *instruction*.
- Storing and copying text (*strings*) using *bytes* and *ASCII*.
- Turning a **RISC-V instruction** into its **32-bit machine code**, and *back again*.
- *Loading* any **32-bit** *constant* with **two** instructions.
- Working out the numbers stored inside *branch instructions*

I used Claude Opus 5.5 to help me format the LaTeX code. All contents were produced, edited, or verified by me.

[0]Himon Thakur | hthakur@uccs.edu | CC BY-NC 4.0

# 1. Bits Mean Nothing (Until You Decide How to Read Them)

> A computer stores only 0s and 1s. The *same* pattern can be a number, a letter, or an instruction. What it means depends on **which rule you use to read it** and **where it is used**.

Think of the English word *"bat"*: it could be an animal or a piece of sports gear. You only know which from context. Bits work the same way.

Imagine a processor encounters this 32-bit pattern in memory:

`1000 0010 0000 0100 0000 1100 0110 0011`

Is it a large number? A negative fraction? Four characters in a word? Or a machine command?

**As an Integer:** It represents the two's complement integer −2,113,663,897.
**As ASCII Characters:** It represents four 8-bit text characters.
**As a Machine Instruction:** It tells the processor to perform a conditional branch (`bge x8, x4, offset`).

**Exercise 1**

Fill out the table below for the nunber: `1000 0010 0000 0100 0000 1100 0110 0011` (hex `0x82040C63`)

| Read it as... | You get... |
|---|---|
| an unsigned whole number | ??? |
| a signed number (two's complement) | ??? |
| a decimal (floating-point) number | ??? |
| a RISC-V instruction | ??? |

# 2. Registers (Are Storage Bins)

RISC-V (RV32I) provides 32 general-purpose (as in *reusable*) registers named `x0` through `x31`. Because raw numbers are hard to remember, software tools use standard Application Binary Interface (ABI) names:

| Register | Nickname | Used for | Must a function put it back? |
|---|---|---|---|
| `x0` | `zero` | always 0 (writes are ignored) | - |
| `x1` | `ra` | return address | no |
| `x2` | `sp` | stack pointer | **yes** |
| `x3` | `gp` | global pointer | - |
| `x4` | `tp` | thread pointer | - |
| `x5` | `t0` | temporary | no |
| `x6 - x7` | `t1 - t2` | temporaries | no |
| `x8` | `s0/fp` | saved / frame pointer | **yes** |
| `x9` | `s1` | saved | **yes** |
| `x10 - x11` | `a0 - a1` | arguments *and* return value | no |
| `x12 - x17` | `a2 - a7` | more arguments | no |
| `x18 - x27` | `s2 - s11` | saved | **yes** |
| `x28 - x31` | `t3 - t6` | temporaries | no |

> **Remember**: **t** = *temporary* (anyone may overwrite it); **s** = *saved* (if you use it, restore it before you return); **a** = *argument*.

**Rules of Preserved Registers:**

- **Caller-saved (`t0 - t6`, `a0 - a7`):** If a calling function cares about values stored here, it must save them before invoking another routine. The called routine may overwrite them freely.
- **Callee-saved (`s0 - s11`, `sp`):** A subroutine is obligated to return these registers exactly as it found them. If it needs to use `s2`, it must first push `s2` onto the stack and pop it off before returning.

**Exercise 2**

A function wants to keep a loop counter safe while it calls another function. Should it use `t0` or `s1`? Why?

# 3. Text Through Characters & Strings

Computers store characters as numbers. Standard **ASCII** uses **7 bits** to define **128** distinct characters (0 to 127), stored inside an **8-bit byte** (with the top bit set to 0).

A string is simply an array of *contiguous* **1-byte ASCII** characters in memory terminated by the special null character '`\0`' (value 0). When scanning a string, the CPU loops until it loads a byte with a numerical value of **zero**. It works like a period at the end of a sentence.

**Exercise 3**

The bits `0x00000048` are sitting in memory. Give **two** things they could mean.(Hint: look up `0x48` in the ASCII table.

## 3.1. ASCII: a lookup table from letters to numbers

Computers cannot store the letter `A` directly, so we agree on a code:

**ASCII** gives each of **128** symbols a number from 0 to 127. That fits in **7 bits**, and each character is stored in one **byte** (*8 bits*) with the top bit set to 0.

| Char | Dec | Hex | Char | Dec | Hex |
|---|---|---|---|---|---|
| `NUL` (\0) | 0 | `0x00` | `A` | 65 | `0x41` |
| ... | ... | ... | ... | ... | ... |
| space | 32 | `0x20` | `H` | 72 | `0x48` |
| ... | ... | ... | ... | ... | ... |
| `!` | 33 | `0x21` | `Z` | 90 | `0x5A` |
| ... | ... | ... | ... | ... | ... |
| `0` | 48 | `0x30` | `a` | 97 | `0x61` |
| ... | ... | ... | ... | ... | ... |
| `9` | 57 | `0x39` | `z` | 122 | `0x7A` |
| ... | ... | ... | ... | ... | ... |

**A trick**:
**Lowercase → uppercase:** subtract 32.

$$\text{“}\mathbf{h}\text{"}(104) - 32 = 72 = \text{“}\mathbf{H}\text{"}$$

**Digit character → number:** subtract `'0'` (48).

$$\text{“}\mathbf{7}\text{"}(55) - 48 = 7$$

## 3.2. Strings: bytes in a row, ending with zero

**Example 1**

[The string `"CAT"` in memory]

| **Address** | base + 0 | base + 1 | base + 2 | base + 3 |
|---|---|---|---|---|
| **Value** | `0x43` (`C`) | `0x41` (`A`) | `0x54` (`T`) | `0x00` (end) |

3 letters, but **4 bytes** of memory.

Because each character is one byte, use the **byte** instructions:

`lbu` (load byte, unsigned) and `sb` (store byte).

Using `lw`/`sw` would move four characters at once.

**Exercise 4**

Write the bytes (in hex) for the string `"Hi!"`. How many bytes does it use?

### 3.3. Worked program: copying a string (`strcpy`)

**Goal**: copy string `y` into string `x`, one character at a time, stopping after the 0 byte.

```
void strcpy(char x[], char y[]) {
    int i = 0;
    while ((x[i] = y[i]) != '\0')   // copy; stop after end marker
        i += 1;
}
```

**Setup**: address of `x` is in `a0`, address of `y` is in `a1`, and we keep `i` in `s2`.

```
strcpy:
    addi  sp, sp, -4      # make room on the stack
    sw    s2, 0(sp)       # save s2 ("s" regs must be restored)
    add   s2, x0, x0      # i = 0
L1:
    add   t0, s2, a1      # t0 = address of y[i]
    lbu   t1, 0(t0)       # t1 = y[i]           (one byte)
    add   t2, s2, a0      # t2 = address of x[i]
    sb    t1, 0(t2)       # x[i] = y[i]
    beq   t1, x0, L2      # if that byte was 0, we're done
    addi  s2, s2, 1       # i = i + 1  (1 byte per char)
    jal   x0, L1          # go around again
L2:
    lw    s2, 0(sp)       # restore s2
    addi  sp, sp, 4       # give the stack space back
    jalr  x0, 0(ra)       # return
```

**Example 2**

[Tracing `strcpy` with `y = "Hi"`]

| `i` (`s2`) | byte loaded (`t1`) | written to | loop again? |
|---|---|---|---|
| 0 | `0x48` (`H`) | `x[0]` | yes |
| 1 | `0x69` (`i`) | `x[1]` | yes |
| 2 | `0x00` (end) | `x[2]` | **no** → L2 |

Notice the end marker is copied *before* the loop stops, so `x` is a proper string too.

**Exercise 5**

Why does `strcpy` save and restore `s2` but not `t0`, `t1`, `t2`?

**Exercise 6**

Rewrite `strcpy` so it keeps `i` in `t3` instead of `s2`. Which four instructions can you delete?

## 4. Instructions Are Numbers Too

**Preserved Registers:**

- **Caller-saved (`t0 - t6`, `a0 - a7`):** If a calling function cares about values stored here, it must save them before invoking another routine. The called routine may overwrite them freely.
- **Callee-saved (`s0 - s11`, `sp`):** A subroutine is legally obligated to return these registers exactly as it found them. If it needs to use `s2`, it must first push `s2` onto the stack and pop it off before returning.

**Color key used below:**

**opcode** **funct** (extra operation bits) **register** **immediate** (constant)

## 4.1. R-type: register ⊕ register → register

Used by `add`, `sub`, and other operations on two registers.

| funct7 | rs2 | rs1 | funct3 | rd | opcode |
|---|---|---|---|---|---|
| 7 bits | 5 bits | 5 bits | 3 bits | 5 bits | 7 bits |

- **opcode**: the general kind of instruction.
- **funct3, funct7**: narrow it down (e.g., *add* vs. *subtract*).
- **rd**: destination register (where the answer goes).
- **rs1, rs2**: the two source registers.

**Example 3**

[Encoding an R-Type Instruction (`add x9, x20, x21`]

**Step 1: Identify the roles of each operand.**

- Destination register `rd` = `x9` = $9_{10}$ = $01001_2$
- First source register `rs1` = `x20` = $20_{10}$ = $10100_2$
- Second source register `rs2` = `x21` = $21_{10}$ = $10101_2$

**Step 2: Look up opcode and function constants.** From the RISC-V specification: `opcode` = $0110011_2$ ($51_{10}$), `funct3` = $000_2$, and `funct7` = $0000000_2$.

**Step 3: Lay out the 32 bits into the R-type template.**

| `funct7` (7 bits) | `rs2` (5 bits) | `rs1` (5 bits) | `funct3` (3 bits) | `rd` (5 bits) | `opcode` (7 bits) |
|---|---|---|---|---|---|
| 0000000 | 10101 | 10100 | 000 | 01001 | 0110011 |

**Step 4: Group into 4-bit nibbles and convert to hexadecimal.**

$$\underbrace{0000}_{0}\ \underbrace{0001}_{1}\ \underbrace{0101}_{5}\ \underbrace{1010}_{A}\ \underbrace{0000}_{0}\ \underbrace{0100}_{4}\ \underbrace{1011}_{B}\ \underbrace{0011}_{3} \Longrightarrow \texttt{0x015A04B3}$$

| 0000000 | 10101 | 10100 | 000 | 01001 | 0110011 |
|---|---|---|---|---|---|
| funct7 | rs2=21 | rs1=20 | funct3 | rd=9 | opcode |

Result: `0x015A04B3`.

**Exercise 7**

Encode `sub x5, x6, x7`. (`sub` has funct7 = `0100000`; everything else is the same as `add`.)

## 4.2. I-type: register ⊕ constant → register (and loads)

R-type has no room for a constant, so instructions like `addi` and `lw` use a different layout. The `funct7` and `rs2` boxes merge into one **12-bit immediate**.

| immediate[11:0] | rs1 | funct3 | rd | opcode |
|---|---|---|---|---|
| 12 bits | 5 bits | 3 bits | 5 bits | 7 bits |

The immediate is stored in **two's complement**, so it can be negative.

**Example 4**

[A practical one: `addi x5, x0, 72` ("put the letter H in x5")]

| 000001001000 | 00000 | 000 | 00101 | 0010011 |
|---|---|---|---|---|
| 72 | rs1=0 | funct3 | rd=5 | addi |

Result: `0x04800293`. Since `x0` is always 0, this just sets `x5` to 72.

**Example 5**

[Wide Immediates with `lui` and `addi`] An I-type instruction only has 12 bits for a constant, which limits values to $[-2048, +2047]$. How do we load a large 32-bit constant like 3,998,976 into register `x19`?

**Step 1: Write the number in hexadecimal.**

$$3{,}998{,}976_{10} = \texttt{0x003D0500} = \underbrace{\texttt{0000 0000 0011 1101 0000}}_{\text{Upper 20 bits: } \texttt{0x003D0}=976_{10}} \quad \underbrace{\texttt{0101 0000 0000}}_{\text{Lower 12 bits: } \texttt{0x500}=1280_{10}}$$

**Step 2: Use `lui` to set the top 20 bits.** `lui x19, 976` loads `0x003D0` into bits $[31:12]$ and automatically clears bits $[11:0]$ to zero.
**Step 3: Use `addi` to add the bottom 12 bits.** `addi x19, x19, 1280` adds 1280 (`0x500`) to `x19`, resulting in the exact value `0x003D0500`.

**Watch out**

**Watch Out for Sign Extension!** Because the 12-bit immediate in `addi` is *signed*, if its highest bit (bit 11) is 1, it acts as a negative value and will subtract from the upper bits! If bit 11 is 1, you must add +1 to your `lui` constant to cancel out the subtraction.

**Exercise 8**

Encode `addi x10, x10, -1`. (Hint: $-1$ in 12-bit two's complement is all ones.)

## 4.3. S-type: stores

A store like `sw x9, 40(x22)` writes a register *to* memory. It needs two registers (the value and the base address) plus an offset, but **no destination register**. So the immediate is **split in two** and uses the empty `rd` slot:

| **imm[11:5]** | **rs2** | **rs1** | **funct3** | **imm[4:0]** | **opcode** |
|---|---|---|---|---|---|
| 7 bits | 5 bits | 5 bits | 3 bits | 5 bits | 7 bits |

**Why split it?** So that rs1, rs2, funct3 and opcode stay in the *same place in every format*. The hardware can then start reading registers before it even knows which format it has.

**Example 6**

[`sw x9, 40(x22)`] $40 =$ `0000001 01000` $\rightarrow$ upper 7 bits `0000001`, lower 5 bits `01000`.

| **0000001** | **01001** | **10110** | **010** | **01000** | **0100011** |
|---|---|---|---|---|---|
| imm[11:5] | rs2=9 | rs1=22 | sw | imm[4:0] | store |

Result: `0x029B2423`. The value being stored (`x9`) goes in **rs2**.

**Exercise 9**

Encode `sb x6, 0(x7)` (funct3 = `000`).

## 4.4. Decoding: going backwards

To read machine code: (1) write it in binary, (2) look at the **last 7 bits** (opcode) to find the format, (3) cut the rest into that format's boxes.

**Example 7**

[Decode `0x00A30333`] Binary: `0000000 01010 00110 000 00110 0110011`. Opcode `0110011` $\rightarrow$ R-type; funct7 and funct3 are all zero $\rightarrow$ `add`. rs2 = 10, rs1 = 6, rd = 6. Answer: `add x6, x6, x10`.

**Exercise 10**

Decode `0xFFC4A303`. (Its opcode `0000011` is a load.)

## 4.5. Quick reference: formats and opcodes

**Color key used below:**

**opcode** **funct** (extra operation bits) **register** **immediate** (constant)

| **Type** | (bit) **31 - 25** | (bit) **24 - 20** | (bit) **19 - 15** | (bit) **14 - 12** | (bit) **11 - 7** | (bit) **6 - 0** | **used for** |
|---|---|---|---|---|---|---|---|
| **R**-type | funct7 | rs2 | rs1 | funct3 | rd | opcode | `add`, `sub` |
| **I**-type | imm[11:0] | | rs1 | funct3 | rd | opcode | `addi`, `lw`, `lbu`, `jalr` |
| **S**-type | imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode | `sw`, `sb` |
| **SB**-type | imm[12-10:5] | rs2 | rs1 | funct3 | imm[4:1-11] | opcode | `beq`, `bne` |
| **U**-type | imm[31:12] | | | | rd | opcode | `lui` |
| **UJ**-type | imm[20-10:1-11-19:12] | | | | rd | opcode | `jal` |

| **Instruction** | **Type** | **funct7** | **funct3** | **opcode** | **Instruction** | **Type** | **funct7** | **funct3** | **opcode** |
|---|---|---|---|---|---|---|---|---|---|
| `add` | R | 0000000 | 000 | 0110011 | `sw` | S | - | 010 | 0100011 |
| `sub` | R | 0100000 | 000 | 0110011 | `sb` | S | - | 000 | 0100011 |
| `addi` | I | - | 000 | 0010011 | `beq` | SB | - | 000 | 1100011 |
| `lw` | I | - | 010 | 0000011 | `bne` | SB | - | 001 | 1100011 |
| `lbu` | I | - | 100 | 0000011 | `lui` | U | - | - | 0110111 |
| `jalr` | I | - | 000 | 1100111 | `jal` | UJ | - | - | 1101111 |

**Why are fields shaped this way?**

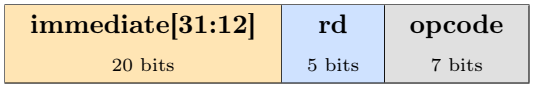
1. **Fixed Register Locations:** Notice that source register 1 (`rs1`) is *always* at bits [19 : 15], and source register 2 (`rs2`) is *always* at bits [24 : 20]. The hardware can read the register values from the register file **immediately**, before it even finishes determining what operation the instruction wants to run!
2. **Split Immediates in S and B Formats:** Store and branch instructions do not write a result to a destination register. Instead of moving `rs2`, the architecture splits the numeric immediate around `rs2` so that `rs2` remains strictly locked at bits [24 : 20].

## 5. Loading big numbers

An I-type immediate has only 12 bits, so `addi` can only add numbers from −2048 to +2047. To load *any* 32-bit number, split it into a **top 20 bits** part and a **bottom 12 bits** part, and use two instructions.

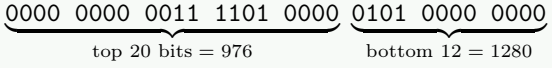
- `lui rd, upper` (*load upper immediate*, U-type) puts `upper` into bits 31-12 and zeros bits 11-0.
- `addi rd, rd, lower` then adds the bottom 12 bits.

U-type:




**Example 8**

[Load 3,998,976 into `x19`] In hex this is `0x003D0500`. Split it:




```
lui   x19, 976          # x19 = 0x003D0000
addi  x19, x19, 1280    # x19 = 0x003D0500 = 3,998,976
```

**Watch out**

`addi` treats its 12 bits as **signed**. If bit 11 of the bottom part is 1 (i.e., the bottom 3 hex digits are `0x800` or more), `addi` will actually *subtract*. Fix: add 1 to the `lui` value.
**Practical example: an orange colour code, `0x00FF8800`.** Bottom part `0x800` has bit 11 set, so it acts as −2048.

```
lui   x5, 0xFF9       # one MORE than 0xFF8
addi  x5, x5, -2048   # 0x00FF9000 - 2048 = 0x00FF8800
```

The `li` pseudo-instruction does this adjustment for you automatically.

**Exercise 11**

Load `0x12345678` into `x5` using `lui` and `addi`.

**Exercise 12**

Load `0x12345ABC` into `x5`. Does the `lui` value need the +1 fix?

## 6. Branches: "go this many bytes forward or back"

**PC-Relative Addressing**: RISC-V conditional branches calculate their jump target relative to the Program Counter.

A branch doesn't store the full address it jumps to (that needs 32 bits and there isn't room). Instead it stores a **distance from where it is right now**. This is called **PC-relative addressing**:

$$\text{new PC} = \text{PC of the branch} + \text{offset}$$

It's like giving directions as "three houses down" instead of a full street address. The directions still work if the whole street is moved somewhere else.

**How to compute the offset:**

$$\text{offset} = \text{address of target} - \text{address of the branch instruction}$$

Positive = jump forward, negative = jump back (loops).

**The Curious Case of the Missing Bit 0:** Because every instruction is aligned on at least a 2-byte boundary, all jump targets are even numbers. Thus, the branch offset's least significant bit (bit 0) is *always* 0. RISC-V simply omits bit 0 from the instruction encoding. By doing so, a 12-bit immediate field can jump twice as far (covering a 13-bit range of ±4 KB)!

**Example 9**

[Loop Calculation Walkthrough]

| Address | Instruction | Action |
|---|---|---|
| 80000 | `Loop: slli x10, x22, 2` | Multiply index by 4 |
| 80004 | `add x10, x10, x25` | Add base address |
| 80008 | `lw x9, 0(x10)` | Load element |
| 80012 | `bne x9, x24, Exit` | **Branch 1: Forward** |
| 80016 | `addi x22, x22, 1` | Increment index |
| 80020 | `beq x0, x0, Loop` | **Branch 2: Backward** |
| 80024 | `Exit: ...` | Next code block |

- **Branch 1 (`bne` at 80012 targeting `Exit` at 80024):**

  $$\text{Offset} = \text{Target} - \text{PC} = 80024 - 80012 = +\mathbf{12} \text{ bytes}$$

  Hardware shifts this right by 1 bit (drops bit 0): $12/2 = \mathbf{6} = 0000\ 0000\ 0110_2$.
- **Branch 2 (`beq` at 80020 targeting `Loop` at 80000):**

  $$\text{Offset} = \text{Target} - \text{PC} = 80000 - 80020 = -\mathbf{20} \text{ bytes}$$

−20 in 13-bit two's complement is `1 1111 1110 1100`$_2$. Dropping bit 0 leaves `1111 1111 0110`$_2$.

## 6.1. How far can a branch reach?

- Instructions always start at even addresses, so the offset's last bit is always 0. RISC-V **doesn't store that bit**, which doubles the reach for free.
- Conditional branch (`beq`, `bne`): about ±4 KiB (exactly −4096 to +4094 bytes).
- `jal`: about ±1 MiB.
- Farther than that? Branch over a `jal`, or use `lui`/`auipc` + `jalr`.

## 6.2. Where the offset bits go (optional detail)

The branch format uses the same box positions as S-type, but the offset bits are shuffled: bits 31–25 hold imm[12] and imm[10:5]; bits 11–7 hold imm[4:1] and imm[11].

**Example 10**

[Filling in `beq x0, x0, -20`] −20 as a 13-bit two's complement number: `1 1111 1110 1100`.

- imm[12] = 1, imm[10:5] = `111111` → bits 31–25 = `1111111`
- imm[4:1] = `0110`, imm[11] = 1 → bits 11–7 = `01101`

| **1111111** | **00000** | **00000** | **000** | **01101** | **1100011** |
|---|---|---|---|---|---|
| imm[12\|10:5] | rs2=0 | rs1=0 | beq | imm[4:1\|11] | branch |

**Exercise 13**

Here is a loop that counts the letters in a string (like `strlen`):

```
1000  Loop:  lbu x6, 0(x10)
1004  beq x6, x0, Done
1008  addi x10, x10, 1
1012  addi x5, x5, 1
1016  jal x0, Loop
1020  Done:
```

What offset is stored in the `beq`? In the `jal`?

**Exercise 14**

A `beq` at address `0x1000` wants to jump to `0x3000`. Can it? Why or why not?

# Recap

**Remember these**

1. Bits have no meaning on their own; the instruction that reads them decides.
2. Characters are 1 byte (ASCII). Strings end with a 0 byte. Use `lbu`/`sb`.
3. Every RISC-V instruction is 32 bits. The **last 7 bits (opcode)** tell you the format.
4. Registers live in the same bit positions in every format: rs1 = 19–15, rs2 = 24–20, rd = 11–7.
5. Assembly order ≠ bit order. `add rd, rs1, rs2` puts rd near the *right* end.
6. Stores have no rd, so their constant is split around it.
7. Big constants: `lui` (top 20 bits) + `addi` (bottom 12). Add 1 to `lui` if bit 11 is set.
8. Branches store *target* − *branch address*, not the full address.

# Answer key for Exercises

**Ex. 1.**
- an unsigned whole number - 2,181,303,395
- a signed number (two's complement) - −2,113,663,901
- a decimal (floating-point) number - about $-9.70 \times 10^{-38}$
- a RISC-V instruction - `beq x8, x0, -4040` (a branch)

**Ex. 2.** `s1`. Saved registers are guaranteed to survive a function call; the called function may overwrite `t0`.

**Ex. 3.** The number 72, or the ASCII letter `H` (followed by zero bytes, i.e., the string `"H"`). Which one depends on the code that reads it.

**Ex. 4.** `0x48 0x69 0x21 0x00` – 4 bytes (3 characters + end marker).

**Ex. 5.** `s2` is a saved register, so `strcpy` must return it unchanged. `t` registers are temporaries; the caller doesn't expect them to survive.

**Ex. 6.** Replace every `s2` with `t3`, then delete `addi sp,sp,-4`, `sw s2,0(sp)`, `lw s2,0(sp)`, `addi sp,sp,4`.

**Ex. 7.** `0100000 | 00111 | 00110 | 000 | 00101 | 0110011` → `0x407302B3`.

**Ex. 8.** `111111111111 | 01010 | 000 | 01010 | 0010011` → `0xFFF50513`.

**Ex. 9.** `0000000 | 00110 | 00111 | 000 | 00000 | 0100011` → `0x00638023`.

**Ex. 10.** `111111111100 | 01001 | 010 | 00110 | 0000011`: imm = −4, rs1 = 9, funct3 `010` = `lw`, rd = 6 → `lw x6, -4(x9)`.

**Ex. 11.** Bottom `0x678` has bit 11 = 0, no fix needed: `lui x5, 0x12345` then `addi x5, x5, 0x678` (= 1656).

**Ex. 12.** Yes. Bottom `0xABC` has bit 11 = 1, so it acts as `0xABC` − 4096 = −1348. Use `lui x5, 0x12346` then `addi x5, x5, -1348`. Check: `0x12346000` − 1348 = `0x12345ABC`.

**Ex. 13.** `beq`: 1020 − 1004 = +16 (machine code `0x00030863`). `jal`: 1000 − 1016 = −16 (machine code `0xFF1FF06F`).

**Ex. 14.** No. The distance is `0x2000` = 8192 bytes, beyond the ±4 KiB branch range. Branch over a `jal` instead (reach ±1 MiB).

# Student Practice Worksheet

## Part 1: Concept & Format Recognition

**Q1.** Match each instruction below with its correct RISC-V format (**R, I, S, B, U,** or **J**):

- (a) `sub x4, x5, x6`: _____
- (b) `lw x10, 16(sp)`: _____
- (c) `sb x7, 0(a1)`: _____
- (d) `bge x1, x2, Loop`: _____
- (e) `lui x12, 0x10000`: _____
- (f) `jal ra, MyFunc`: _____

**Q2.** Why does RISC-V split the immediate field into two separate chunks in Store (**S**) instructions instead of keeping the 12 bits consecutive at the beginning of the instruction word?

**Q3.** In the standard string copying function (`strcpy`), explain why the loop loads data using `lbu` (Load Byte Unsigned) rather than `lw` (Load Word).

## Part 2: Encoding Calculations

**Q4.** Encode the instruction `sub x5, x6, x7` into binary and hexadecimal machine code.

- Given: `sub` has `opcode` = `0110011`, `funct3` = `000`, `funct7` = `0100000`.
- Register numbers: `x5` ($5_{10}$), `x6` ($6_{10}$), `x7` ($7_{10}$).

| funct7 | rs2 | rs1 | funct3 | rd | opcode |
|---|---|---|---|---|---|
| | | | | | |

Hexadecimal representation: `0x`_____

**Q5.** Encode the instruction `lw x9, 32(x22)` into machine code.

- Given: `lw` has `opcode` = `0000011`, `funct3` = `010`.
- Immediate: $32_{10}$. Registers: destination `x9`, base register `x22`.

Hexadecimal representation: `0x`_____

## Part 3: Wide Immediates & Branch Offsets

**Q6.** You want to load the constant `0x0042F820` into register `x11`.

- Notice that the lower 12 bits are `0x820`. What is the value of bit 11? _____
- Will `addi` sign-extend this as a positive or negative number? _____
- Write the correct two-instruction sequence (`lui` followed by `addi`) that loads the desired constant without sign-extension corruption:

```
lui  x11, _____
addi x11, x11, _____
```

**Q7.** A conditional branch instruction `beq x10, x11, Target` is located at memory address `0x00010040`. The label `Target` is located at address `0x00010078`.

- Calculate the byte offset in decimal: _____ bytes.
- Calculate the numeric value stored inside the 12-bit immediate field of the branch instruction: _____.

# Step-by-Step Solutions & Explanations

**Q1: Format Matching**

- (a) `sub`: **R-type** (Three registers: `rd`, `rs1`, `rs2`).
- (b) `lw`: **I-type** (Load instructions take a base register and a 12-bit offset).
- (c) `sb`: **S-type** (Store byte takes source data register, base address, and offset).
- (d) `bge`: **B-type** (Conditional branch comparing two registers with PC-relative offset).
- (e) `lui`: **U-type** (Upper immediate format carrying a 20-bit payload).
- (f) `jal`: **J-type** (Unconditional jump storing return address in `ra`).

**Q2: Why split the immediate in S-type instructions? Answer:** To maintain hardware decode consistency. By splitting the immediate into bits $[31:25]$ and $[11:7]$, the source registers `rs1` (bits $[19:15]$) and `rs2` (bits $[24:20]$) remain at the exact same physical bit positions as in R-type instructions. The CPU can immediately read both register values in parallel without waiting to decode the full instruction opcode.

**Q3: Why `lbu` and `sb` in `strcpy`? Answer:** Characters in standard ASCII strings occupy exactly 1 byte (8 bits). A word in RV32 is 4 bytes. If the code used `lw` or `sw`, it would read and write 4 characters at once, corrupting neighboring data in memory and overshooting the null-terminator byte.

**Q4: Machine Code for `sub x5, x6, x7`**

- `funct7` $= 0100000_2$, `rs2` $=$ `x7` $= 00111_2$, `rs1` $=$ `x6` $= 00110_2$, `funct3` $= 000_2$, `rd` $=$ `x5` $= 00101_2$, `opcode` $= 0110011_2$.
- Binary: `0100000 00111 00110 000 00101 0110011`
- Grouped nibbles: `0100 0000 0111 0011 0000 0010 1011 0011` $\implies$ `0x407302B3`.

**Q5: Machine Code for `lw x9, 32(x22)`**

- `imm[11:0]` $= 32_{10} =$ `0000 0010 0000`$_2$, `rs1` $=$ `x22` $= 22_{10} = 10110_2$, `funct3` $= 010_2$, `rd` $=$ `x9` $= 9_{10} = 01001_2$, `opcode` $= 0000011_2$.
- Binary: `000000100000 10110 010 01001 0000011`
- Grouped nibbles: `0000 0010 0000 1011 0010 0100 1000 0011` $\implies$ `0x020B2483`.

**Q6: Wide Immediate with Sign Extension (`0x0042F820`)**

- The lower 12 bits are `0x820` $=$ `1000 0010 0000`$_2$. Bit 11 is **1**.
- When `addi` executes, it treats bit 11 as the sign bit and sign-extends it to a 32-bit negative number: `0xFFFFF820` $= -2016_{10}$.
- If we simply loaded the upper bits `0x0042F`, the addition would subtract from our upper bits! To compensate, we must add 1 to the upper 20 bits:

$$\texttt{Upper adjusted} = \texttt{0x0042F} + 1 = \texttt{0x00430}$$

- Correct instruction sequence:

```
lui  x11, 0x00430
addi x11, x11, 0x820
```

**Q7: Branch Offset Calculation**

- Target Address $=$ `0x00010078`, Branch PC $=$ `0x00010040`.
- Byte Offset $=$ `0x00010078` $-$ `0x00010040` $=$ `0x38` $= +$**56** bytes.
- Encoded field value: Because bit 0 is omitted (shifted right by 1), the immediate stored in the instruction word is $56/2 =$ **28** (`0x1C`).